

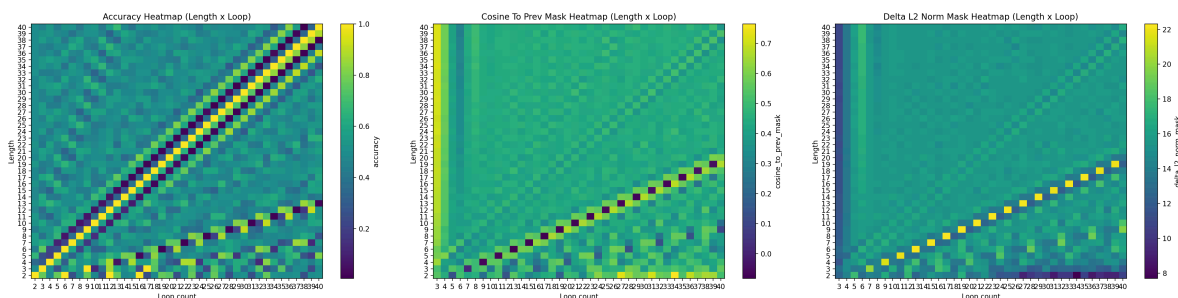
Loop Train & Eval 2026.2.1

默认训练设置：

1. 输入一定长度的任务序列，经过L次loop之后对答案位和padding位decode，然后计算loss
2. 在length=loop的时候decode并计算loss
3. 训练使用curriculum的策略，训练随机使用 $\leq \text{current_length}$ 的任务序列进行训练
4. 模型有causal mask, NoPE

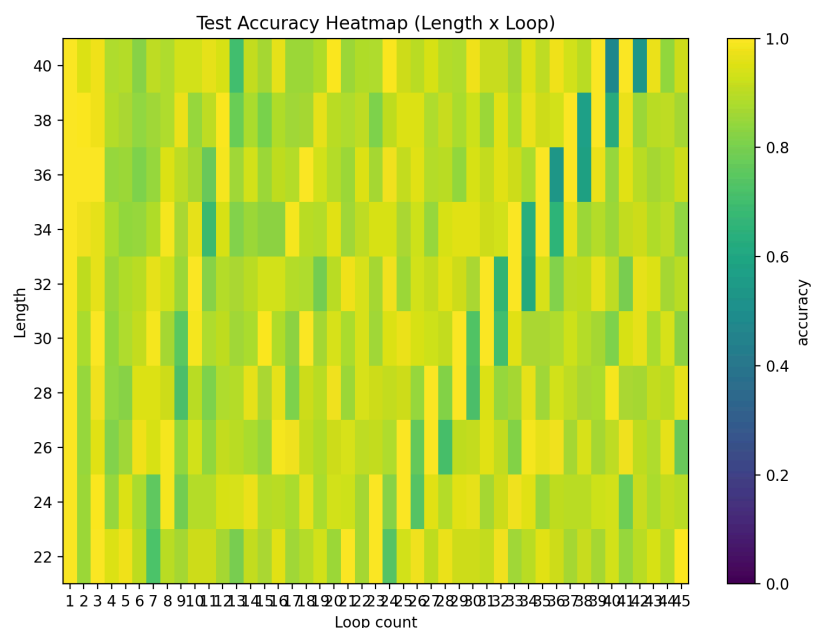
Parity

在2-20的长度上使用默认方式训练，并在更长的序列上测试，发现可以完美自适应泛化



但是发现其hidden无法收敛，且在loop次数=2length+1的时候，相比于前一次loop的hidden几乎正交

尝试prob每次loop结束的hidden，以length=loop为标签，对一个probing head进行训练

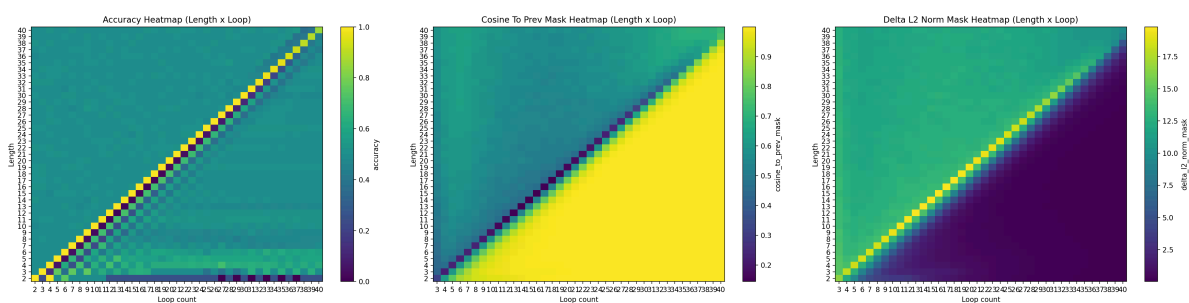


在训练集上达到93%的正确率的时候，测试集在loop=length的位置表现不佳

Length	Acc
22	0.866000
24	0.826000
26	0.762000
28	0.818000
30	0.730000
32	0.669000
34	0.632000
36	0.524000
38	0.568000
40	0.477000

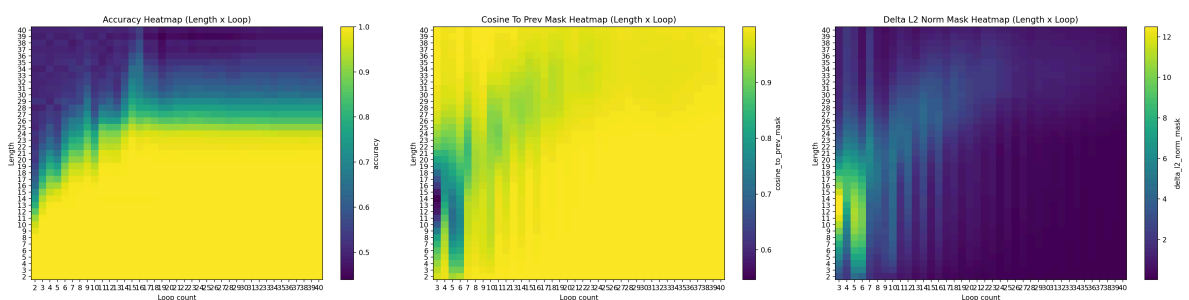
Substep Supervision

在对第k次loop添加前k个字序列parity结果的supervision之后，发现模型收敛，但是仍然只能在length=loop的位置回答正确，这是因为在loop=L+1的时候会有一次正交的变化，把答案翻转，然后迅速收敛，所以后面几乎全都做不对



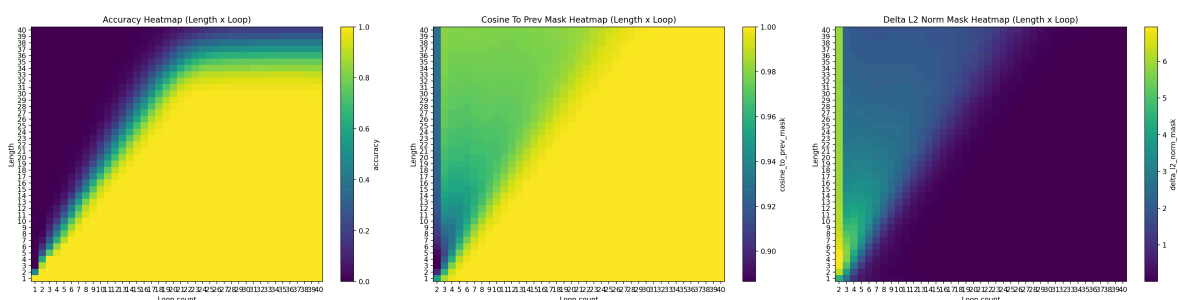
Random Loop

在默认训练方式的基础上，不再固定loop次数，而是对每一个batch随机均匀采样[2-20]的一个loop数，训练得到模型泛化性最差，但是初步学到了长度泛化+收敛的状态

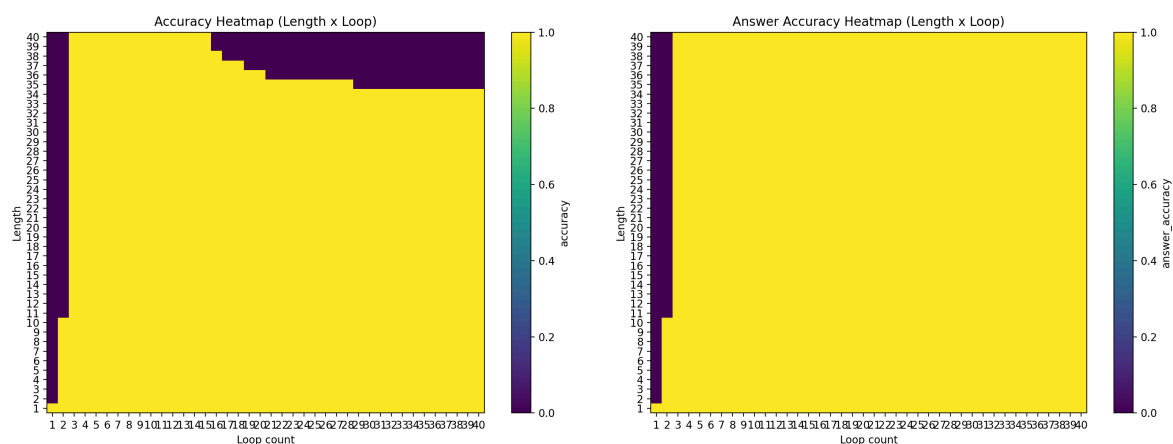


Copy

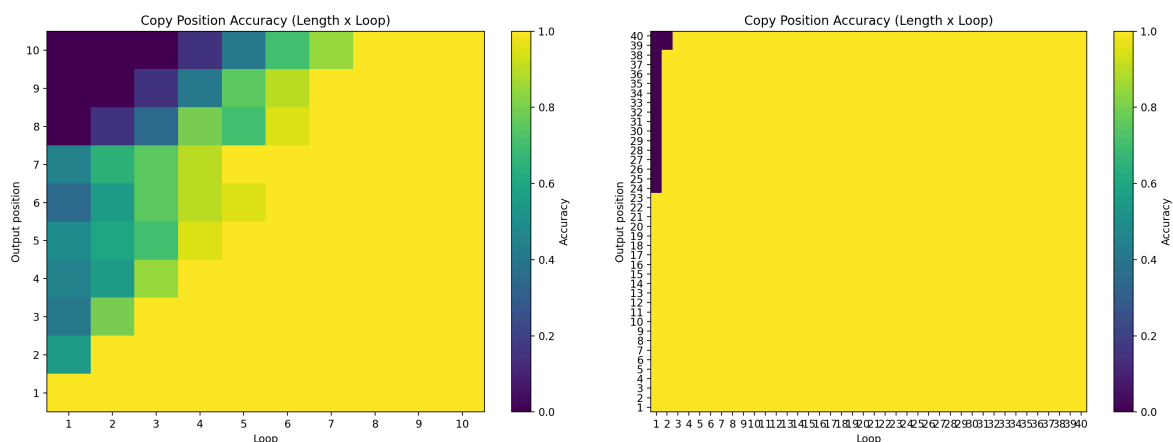
在2-20长度的01序列copy任务上训练，并在20-40上测试长度泛化，发现用默认的训练方式就能得到一个比较好的随长度增加loop，且在length=loop之后直接收敛并保持正确率的解



为了进一步测试其在OOD上的泛化能力，测试在Bernoulli(0.2) Bernoulli(0.1)上的结果，和Bernoulli(0.5)的结果类似，后考虑极端情况，对于全0序列的泛化能力，发现他也能泛化，但是收敛速度会快非常多，且当loop比较久之后，他会在padding的位置翻转一个token，导致如果把padding位计入正确率的话，正确率会直接掉到0，如果单纯看答案的正确率（右图），会发现他还是100%正确率



为了看他是不是真的是一步一步copy的，我看了对于一个固定长度的序列，他的每个位置的正确率随着loop次数的变化，可以看到基本上是一步一步操作的，右图是全0序列的图，也是收敛速度非常快



Binary Addition

考虑二进制加法

Binary Addition. Performing binary addition of two binary numbers with the same number of digits, and the output has one more digit (without removing leading 0 if it appears). Example input:

1	0	+	1	1	>	#	#	#
---	---	---	---	---	---	---	---	---

, example output:

*	*	*	*	*	1	0	1	#	#
---	---	---	---	---	---	---	---	---	---

. We highlight that we do not reverse the output like recent works (Lee et al., 2024; McLeish et al., 2024; Zhou et al., 2024b). We define the length of the problem to be the number of digits to be added, set T to be the same as the problem length, and train with length $[1, 20)$. It has been shown that binary addition without index hint is hard to generalize in vanilla NTP (Zhou et al., 2024a).

用相同的训练方法得到了一个基本上也是能收敛+泛化好的结果，感觉上和copy任务类似



Binary Sum Reverse

Binary Sum. Calculating the sum of the binary string in the binary form (in reversed order). Example input:

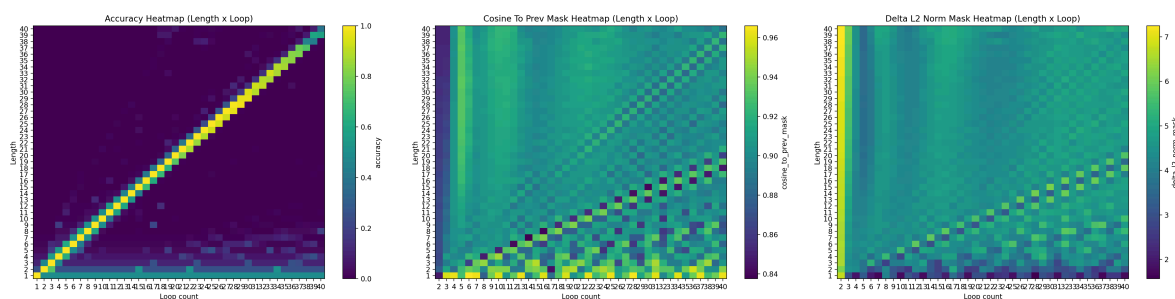
1	0	1	1	>	#	#	#	#
---	---	---	---	---	---	---	---	---

, example output:

*	*	*	*	1	1	#	#	#
---	---	---	---	---	---	---	---	---

. We define the length of the problem to be the number of binary digits to be added, set T to be the same as the problem length, and train with length $[1, 20)$.

这个看起来像是parity任务的升级版，把parity后面缺失的计算结果给补全了，用默认的训练方式得到和parity非常类似的结果，只能在length=loop的时候泛化，且不收敛，能观察到在loop=2L的位置也有类似parity的转向，但是这次只是相对的角度大一点，cosine有0.8左右



Modular addition 11

使用L个0-10的数相加对11取模，在1-20上训练，在20-40上测试，能够完美泛化，且泛化的方式和parity很类似，但是他在length较低loop较大的时候会收敛，这个过程是缓慢的，而不是parity那种剧烈的反转然后收敛，所以在收敛后的正确率仍然很低

